# ELP USB Camera

# Multi-channel Global Synchronization SDK Development Manual

**2026-6-25 Ver.B**

**ELP**

www.elpcctv.com

## Version Update Notes

| Version No. | Modification Date | Modification Description |
|---|---|---|
| **A** | 2026-5-11 | Initial draft established |
| **B** | 2026-6-25 | SDK version upgraded to v1.1.3<br>QT section removed |

# Table of Contents

**VERSION UPDATE NOTES** ..... 错误！未定义书签。

**1.** Overview ..... 错误！未定义书签。

1.1 Purpose of This Document ..... 3

1.2 Introduction to ELP SDK ..... 3

**2.** Setting Up the Development Environment ..... 错误！未定义书签。

2.1 VS2022Pproject ..... 4

**3. Structure Description** ..... 错误！未定义书签。

3.1 Image Format Enumeration ..... 8

3.2 Device Information ..... 8

3.3 Image Format Information ..... 8

3.4 Parsed Image Information ..... 8

3.5 Multi-channel Synchronized Image Grouping ..... 9

3.6 IMU and Magnetometer Information ..... 9

**4. Interface Function Description** ..... 错误！未定义书签。

4.1 SYNC_Initialize ..... 9

4.2 SYNC_Release ..... 10

4.3 SYNC_GetVersion ..... 10

4.4 SYNC_GetLastError ..... 10

4.5 SYNC_GetErrorText ..... 10

4.6 SYNC_EnumDevices ..... 11

4.7 SYNC_OpenDevice ..... 11

4.8 SYNC_CloseDevice ..... 11

4.9 SYNC_GetDeviceFormats ..... 12

4.10 SYNC_SetDeviceFormat ..... 12

4.11 SYNC_GetDeviceFormatIndex ..... 12

4.12 SYNC_SetImageCallback ..... 13

4.13 SYNC_SetCaptureFormat(CamFormat format); ..... 13

4.14 SYNC_StartCapture ..... 14

4.15 SYNC_StopCapture ..... 14

4.16 SYNC_IsCapturing ..... 14

4.17 SYNC_SaveToJPG ..... 14

4.18 SYNC_GetFrameRate ..... 15

4.19 SYNC_ IsOpening ..... 15

4.20 Callback Function Pointer ..... 15

# 1. Overview

## 1.1 Purpose of This Document

This document introduces the API usage of ELP's multi-channel Camera synchronization Windows SDK (hereinafter referred to as the SDK).

Developers can use the SDK under the Windows system to develop software for synchronously acquiring images from multiple USB Cameras, without needing to concern themselves with the underlying implementation. By simply calling a small number of APIs, they can obtain exposure timestamps of multiple devices, timestamped IMU data, and image data (NV12 format). Image frame synchronization can be performed among multiple devices, and image data and IMU data of a single device can be synchronized. The synchronized image data can be directly used for image processing with OpenCV or for video encoding.

This SDK is only applicable to ELP's global shutter USB3.0 Cameras with timestamp functionality (solutions such as SC133HGS, AR0234, OS05B10, etc.).

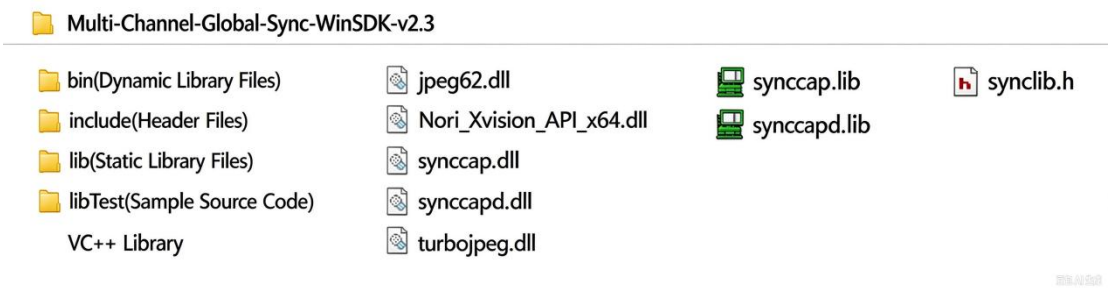
## 1.2 Introduction to ELP SDK

The SDK is applicable to Windows 10 and above.

The SDK is developed in C/C++ and can also be called via Python.

The SDK contents include static library files, dynamic library files, header files, and sample source code.

The SDK contents include static library files, dynamic library files, header files, and sample source code.



The static library files include Debug (synccapd.lib) and Release (synccap.lib). Users can replace the corresponding lib in the project according to their version requirements. Static libraries are used during the compilation and linking stage and are added when configuring the linker in the VS project.

The dynamic library files include Debug (synccapd.dll) and Release (synccap.dll). Users can replace the corresponding dll in the project according to their version requirements. Dynamic libraries are linked during program execution, so before running the program, a copy of the dynamic library must be placed in the directory where the executable program (with the .exe extension) is located; otherwise, the program will fail to run due to a missing dynamic library. jpeg62.dll and turbojpeg.dll are third-party JPEG decoding libraries, and these third-party libraries also need to be copied to the directory where the executable program (with the .exe extension) is located before running the program.

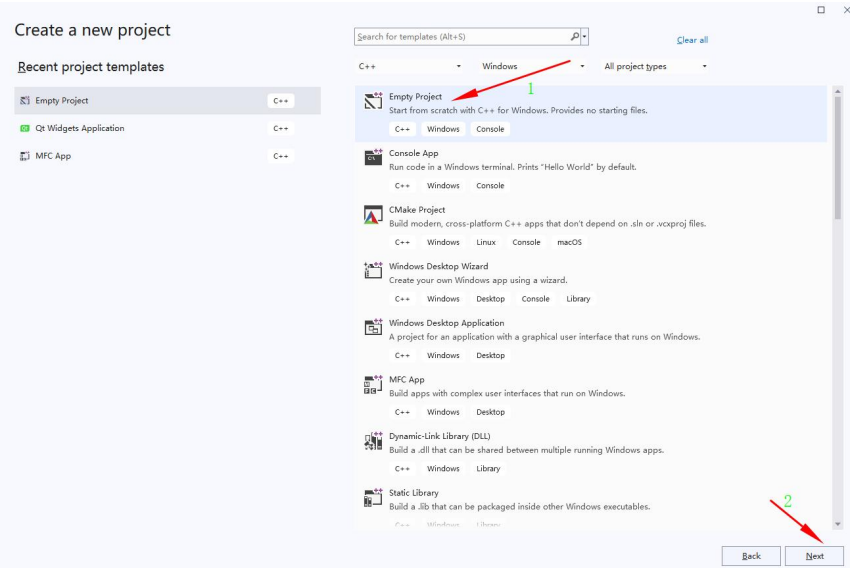
The header file is synclib.h. In the code, use #include "synclib.h" to import the API declarations.

libTest is the sample source code, which contains the source code for all API usages covered in this SDK document.

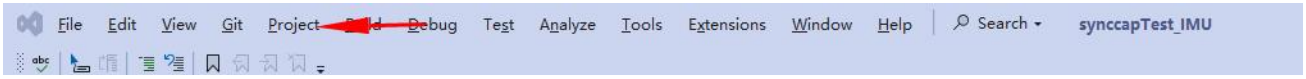
# 2. Setting Up the Development Environment

## 2.1 VS2022 project

**Create an empty project.**



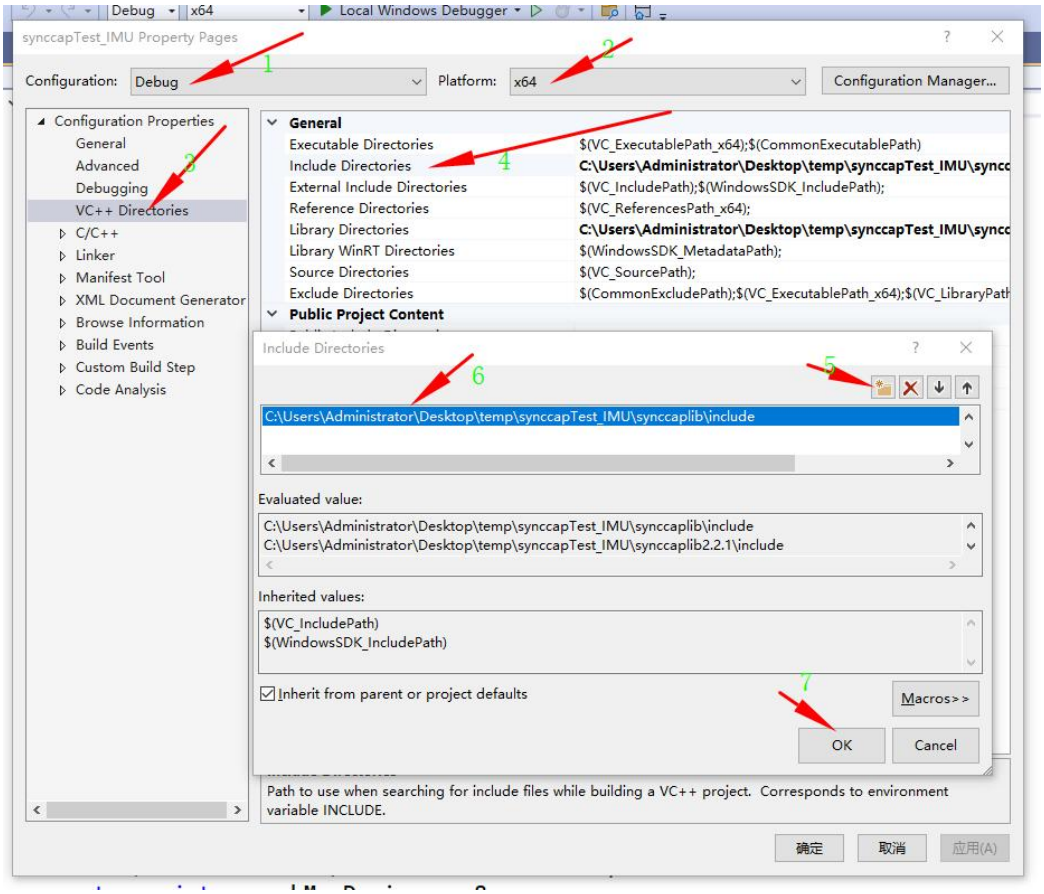
Configure the environment.
Click "Project" in the menu bar.




Click "Properties" to enter the Properties page.
For "Configuration", select Debug or Release; here, select Debug.
For "Platform", select the x64 platform.

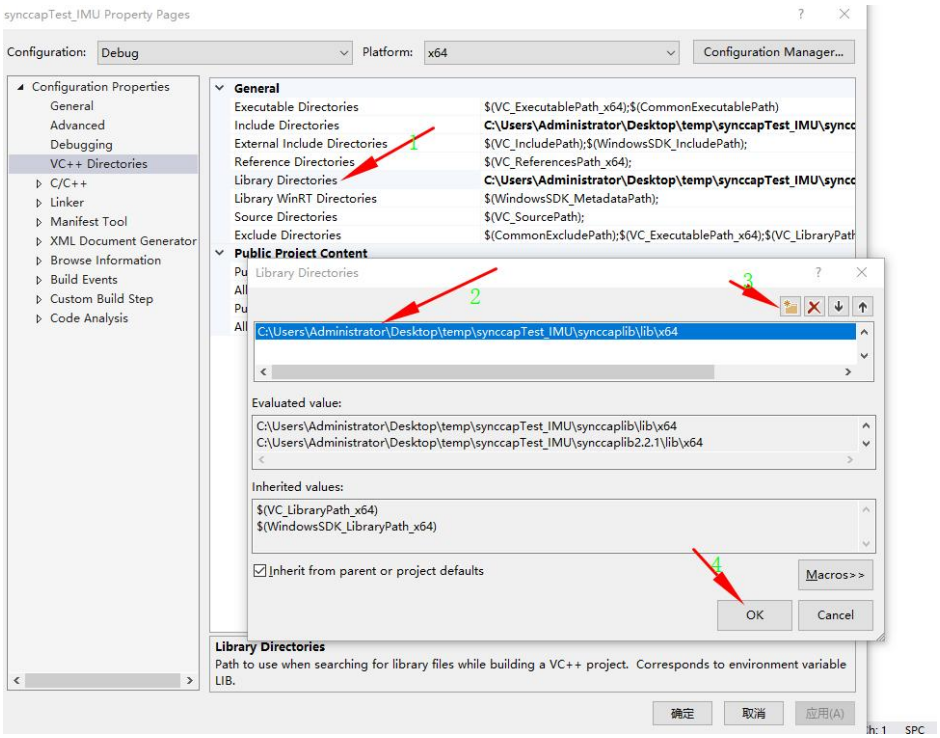
**Configure the header file path**

Click "Configuration Properties" > "VC++ Directories" > "Include Directories" in sequence on the left, add the path where the synclib.h file is located, and click OK, as shown in the figure below. (Alternatively, you can directly add synclib.h to the project.)



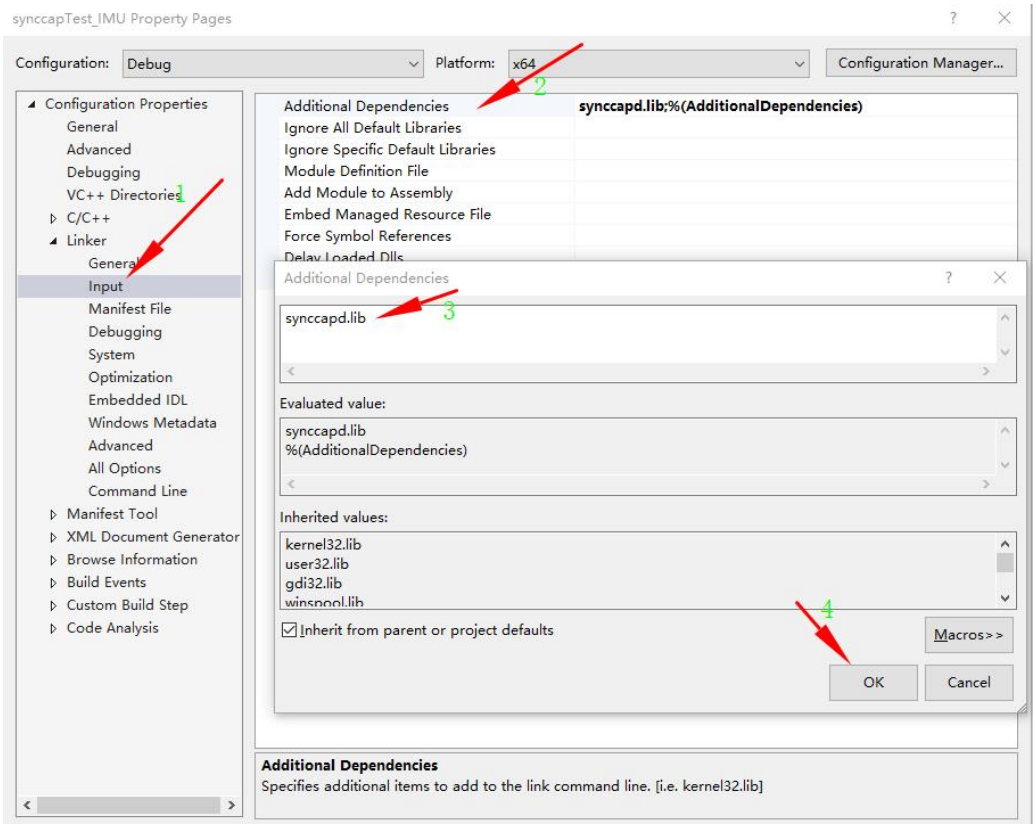

**Configure the static library directory.**

Continuing in VC++ Directories, click "Library Directories", add the SDK's lib directory (note the version matching; here it is x64, replace it with the actual version), and click OK, as shown in the figure below.



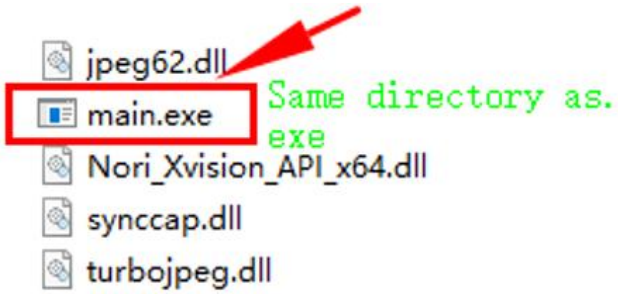

**Link the static library.**

Click "Linker" -> "Input" -> "Additional Dependencies" in sequence on the left of the Properties page, enter the lib to be linked, here it is synclib.lib (note the version; here it is x64, Debug mode, replace it with the actual static library), as shown in the figure below.



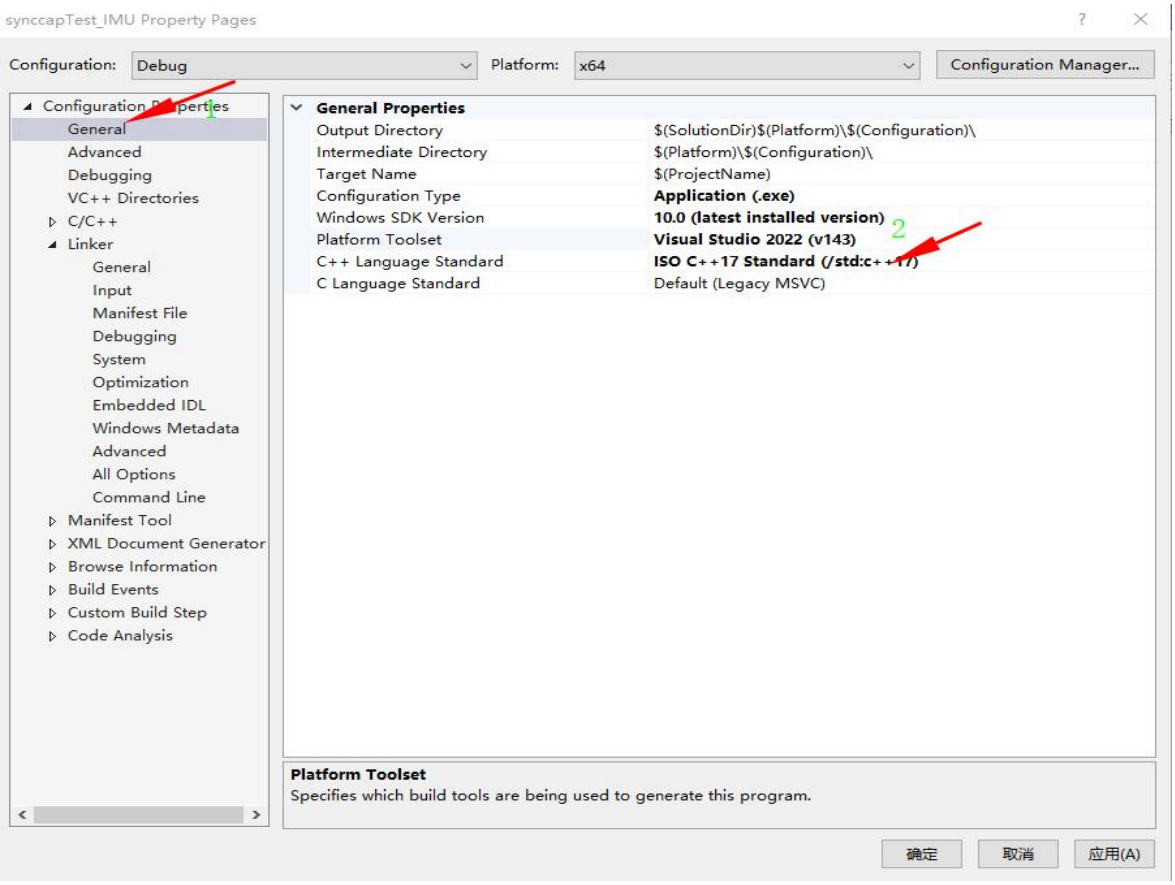
**Import the dynamic library.**

Copy a copy of the .dll files from the synclib/bin directory to the same directory as the project's executable program, paying attention to the version and configuration, as shown in the figure below (x64, Debug mode).




**Set the C++ standard.**

In the Properties page, click "General" -> "C++ Standard" in sequence, and select the C++17 standard, as shown in the figure below.



# 3. Structure Description

## 3.1 Image Format Enumeration

```
typedef enum CamFormat {
  FORMAT_UNKNOWN = 0,
  FORMAT_MJPG = 1,
  FORMAT_RGB24 = 2,
  FORMAT_YUV422 = 3,
  FORMAT_NV12 = 4
}CamFormat;
```

## 3.2 Device Information

```
typedef struct DeviceInfo {
  char name[256];
  char vidpid[16];
  uint32_t number;
}DeviceInfo;
```

## 3.3 Image Format Information

```
typedef struct FormatInfo {
  CamFormat fmt;
  int width;
  int height;
  int fps;
}FormatInfo;
```

## 3.4 Parsed Image Information

```
typedef struct CamData {
  uint8_t* data;
  int width;
  int height;
  int bufSize;
  CamFormat format;
  uint64_t startExpouseTime;
  uint64_t endExpouseTime;
  sICM42688_XYZ_float imu_data[11];
  sAK09940_XYZ_int mtt_data[5];
  int deviceIdx;
  bool isValid;
```

```
}CamData;
```

## 3.5 Multi-channel Synchronized Image Grouping

```
  typedef struct CamGroupData {
    struct CamData camData[8];
    int  deviceCount;
}CamGroupData;
```

## 3.6 IMU and Magnetometer Information

```
  typedef struct {
    uint64_t  uTime;
    float     fAccData_X;
    float     fAccData_Y;
    float     fAccData_Z;
    float     fGyroData_X;
    float     fGyroData_Y;
    float     fGyroData_Z;
}sICM42688_XYZ_float;

  typedef struct sAK09940_XYZ_int {
    uint64_t    uTime;
    int32_t     iX;
    int32_t     iY;
    int32_t     iZ;
    float      Temp;
    int32_t     iStatusBit;
}sAK09940_XYZ_int;
```

# 4. Interface Function Description

## 4.1 SYNC_Initialize

| Function Prototype: int SYNC_Initialize(); |
| --- |
| Function: Initializes SDK resources<br>Parameters: None.<br>Return Value: Returns 0 on success, -1 on failure.<br>Usage Notes: This function must be called to initialize before calling any other API. |

## 4.2 SYNC_Release

Function Prototype: void SYNC_Release();

Function: Releases SDK resources.

Parameters: None.

Return Value: None.

Usage Notes: Call this function when exiting the program.

## 4.3 SYNC_GetVersion

Function Prototype: const char* SYNC_GetVersion();

Function: Gets the SDK version information.

Parameters: None.

Return Value: Returns the version information string.

Usage Notes: None.

## 4.4 SYNC_GetLastError

Function Prototype: const int SYNC_GetLastError();

Function: Gets the error code.

Parameters: None.

Return Value: Returns 0 if there is no error; returns an integer error code if there is an error.

Usage Notes: None.

## 4.5 SYNC_GetErrorText

Function Prototype: const char* SYNC_GetErrorText(int errorCode);

Function: Gets detailed error information.

Parameters: errorCode: Integer error code.

Return Value: Returns the string information corresponding to the error code.

Usage Notes: None.

## 4.6 SYNC_EnumDevices

Function Prototype: int SYNC_EnumDevices(DeviceInfo* devices, int maxCount, int* count);

Function: Enumerates device information.

Parameters: devices:DeviceInfo pointer, used to store the detected device information

maxCount: The maximum capacity of devices.devices

count： Integer pointer, stores the actual number of detected devices.

Return Value: Returns 0 on successful enumeration, -1 on failed enumeration.

Usage Notes: None.

## 4.7 SYNC_OpenDevice

Function Prototype: int SYNC_OpenDevice(uint32_t deviceIndex);

Function: Opens a device.

Parameters: deviceIndex: The index of the device to open (Camera01-Camera08 correspond to 0-7 respectively).

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.8 SYNC_CloseDevice

Function Prototype: SYNC_CloseDevice(uint32_t deviceIndex);

Function: Sets the format of the specified device.

Parameters: deviceIndex: The index of the device to close (Camera01-Camera08 correspond to 0-7 respectively).

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.9 SYNC_GetDeviceFormats

Function Prototype: int SYNC_GetDeviceFormats(uint32_t deviceIndex, FormatInfo* formats, int maxCount, int* count);

Function: Gets the device formats.

Parameters: deviceIndex: Device index.

formats: FormatInfo pointer, used to store all formats supported by this device.

maxCount: Integer variable, the maximum capacity of formats.

count: Integer pointer, stores the number of formats obtained.

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.10 SYNC_SetDeviceFormat

Function Prototype: int SYNC_SetDeviceFormat(uint32_t deviceIndex, int formatIndex);

Function: Sets the device format.

Parameters: deviceIndex: Device index.

formatIndex: The index of the format to set (this index corresponds to the index in the formats obtained by calling SYNC_GetDeviceFormats).

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.11 SYNC_GetDeviceFormatIndex

Function Prototype:int SYNC_GetDeviceFormatIndex(uint32_t deviceIndex);

Function: Gets the index of the current format.

Parameters: deviceIndex: Device index.

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

### 4.12 SYNC_SetImageCallback

Function Prototype: int SYNC_SetImageCallback(ImageCallback callback, void* userData);

Function: Registers a callback function, used to bind the function for acquiring image data. This function must be of type ImageCallback, that is, a function whose return value is void and whose parameters are (const CamGroupData* image, void* userData).

Parameters: callback: Pass in a custom callback function.

userData: The parameter that the user needs to pass in.

Return Value: Returns the current format index on success, -1 on failure.

Usage Notes: Although passing the ImageCallback type as the first parameter of the function is the same as passing a function name whose return value is void and whose parameters are (const CamGroupData* image, void* userData), the advantage of passing the ImageCallback type is that when the user wants to change the function for acquiring image data, they can directly define a function and assign it to an ImageCallback type variable. For example, if the user defines an ImageCallback callBack and a function void fun(const CamGroupData* image, void* userData), assign callBack = fun; this way callback points to the fun function, and calling SetImageCallback(callback, userData) completes the registration, and the user can then acquire images from the fun function. If the user switches to another function void fun2(const CamGroupData* image, void* userData) as the data-acquiring function, they only need to set callback = fun2, without needing to call SetImageCallback again to register. This simplifies the code and makes it more standardized.

## 4.13 SYNC_SetCaptureFormat(CamFormat format);

Function Prototype: void SYNC_SetCaptureFormat(CamFormat format);

Function: Sets the format of the output image.

Parameters: format: The format to set; only FORMAT_NV12 and FORMAT_RGB24 are supported.

Return Value: None.

Usage Notes: None.

## 4.14 SYNC_StartCapture

Function Prototype: int SYNC_StartCapture(int deviceCount);

Function: Starts image capture.

Parameters: deviceCount: Indicates the number of devices participating in image capture. At least deviceCount devices must be opened before calling.

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.15 SYNC_StopCapture

Function Prototype: int SYNC_StopCapture();

Function: Stops image capture.

Parameters: None

Return Value: Returns 0 on success, -1 on failure.

Usage Notes: None.

## 4.16 SYNC_IsCapturing

Function Prototype: bool SYNC_IsCapturing();

Function: Gets the status of whether image capture has started.

Parameters: None

Return Value: Returns true if started, false if stopped.

Usage Notes: None.

## 4.17 SYNC_SaveToJPG

Function Prototype: bool SYNC_SaveToJPG(const unsigned char* rgb, int w, int h, const char* filename, int quality = 85);

Function: Saves a JPG image.

Parameters: rgb: RGB24 image buffer (must be RGB24 data; when saving NV12 format, it needs to be converted to RGB24 format first).

w: Image width.

h: Image height.

filename: Path of the file to save.

quality: Compression quality, default is 85.

Return Value: Returns true on successful save, false on failed save.

Usage Notes: Call this function when a photo needs to be taken.

## 4.18 SYNC_GetFrameRate

Function Prototype: float SYNC_GetFrameRate();

Function: Gets the frame rate of the output image.

Parameters: None.

Return Value: The frame rate value.

Usage Notes: None.

## 4.19 SYNC_ IsOpening

Function Prototype: bool SYNC_IsOpening(uint32_t deviceIndex);

Function: Determines whether the device is open.

Parameters: deviceIndex: Device index.

Return Value: Returns true if the device is open, false if the device is closed.

Usage Notes: None.

## 4.20 Callback Function Pointer

Function Prototype: typedef void (*ImageCallback)(const CamGroupData* image, void* userData);

Function: User-defined callback function, used to obtain image data information from multiple devices.

Parameters: image: A pointer to a group of synchronized image data, internally containing multiple channels of CamData.

userData: The custom parameter passed in by the user when registering the callback,

which will be passed back as-is.

Return Value: None.

Usage Notes: Here, ImageCallback is simply an alias for a function whose return value is void and whose parameters are (const CamGroupData* image, void* userData). ImageCallback represents a pointer to this kind of function, and the two are equivalent. This function is used as the callback function for acquiring image data and needs to be registered by calling SYNC_SetImageCallback().